

Módulo 8 — Ingeniería de Modelos y LLMOps

Ciclo de vida LLM, versionado, fine-tuning vs prompting, observabilidad LangSmith, CI/CD y test-time compute.

BLOQUE 1 · EL CICLO DE VIDA DE LOS MODELOS LLM EN PRODUCCIÓN

Módulo 8 — Ingeniería de Modelos y LLMOps

La operacionalización de los modelos de IA (MLOps para modelos de ML clásico, LLMOps para sistemas basados en LLMs) es la disciplina que cierra la brecha entre los experimentos de data science y los sistemas en producción que crean valor de forma confiable y sostenible. El 87% de los proyectos de IA nunca llegan a producción (VentureBeat). El 40% de los que llegan fallan en el primer año. Las causas no son técnicas (el modelo no funciona) sino operacionales: el sistema se degrada sin monitorización, los costes escalan más rápido de lo previsto, el equipo no tiene visibilidad sobre qué está fallando, y no hay proceso para actualizar el modelo cuando cambia la distribución de los datos.

LLMOps extiende MLOps con las características específicas de los sistemas basados en LLMs: el "modelo" es frecuentemente una API externa (OpenAI, Anthropic, Gemini) que puede cambiar de comportamiento con las actualizaciones del proveedor, el sistema incluye prompts que son tan importantes como el código y deben versionarse con el mismo rigor, la evaluación de calidad es más compleja (texto libre vs categorías predefinidas), y los costes de inferencia son proporcionales al número de tokens procesados y generados.

Versionado de modelos, prompts, y datos

El versionado es la práctica de mantener un historial completo de todos los cambios en el sistema de IA: versiones del modelo, versiones de los prompts, versiones de los datos de entrenamiento y evaluación, y versiones de la configuración del sistema. Sin versionado, es imposible saber qué produjo un cambio de comportamiento observado en producción, reproducir el estado del sistema en un momento anterior, o deshacer un cambio que degrada el rendimiento. Para los LLMs, el versionado de prompts es especialmente crítico: los prompts son parte del código del sistema y su gestión debe incluir control de versiones (git), revisión de cambios (code review), y testing antes de desplegar en producción.

DVC (Data Version Control) es el estándar para el versionado de datasets y modelos de ML: permite hacer commit de referencias a archivos grandes en el repositorio de git mientras almacena los archivos reales en almacenamiento externo (S3, GCS). Los registros de modelos (MLflow Model Registry, Weights & Biases Registry) almacenan los modelos entrenados con metadatos completos (métricas, hiperparámetros, dataset usado) y gestionan el ciclo de vida (staging, production, archived). Para los LLMs que usan APIs de terceros, el versionado incluye el ID del modelo específico de la API (incluyendo versiones con fecha como gpt-4o-2024-08-06) para garantizar la reproducibilidad.

Finetuning vs prompting: la decisión estratégica

La decisión de cuándo usar fine-tuning versus prompting para adaptar un LLM a un caso de uso específico es una de las más importantes en LLMOps. El prompting es más rápido (no requiere datos

de entrenamiento ni tiempo de entrenamiento), más flexible (los prompts se pueden cambiar en minutos), y más económico en el corto plazo. El fine-tuning produce mejor rendimiento para tareas bien definidas con suficientes datos de entrenamiento (>500-1000 ejemplos de alta calidad), reduce el coste de inferencia (el modelo fine-tuneado con instrucciones más cortas puede reemplazar prompts largos con muchos ejemplos), y puede producir comportamientos que no son alcanzables con prompting (estilos muy específicos, formatos de output muy particulares, conocimiento de dominio especializado).

La regla práctica: empieza siempre con prompting. Si después de optimizar el prompt el rendimiento no es suficiente, considera el fine-tuning. El fine-tuning es especialmente valioso para: comportamientos de formato muy específico que requieren muchos ejemplos en el prompt (y el fine-tuning eliminaría esa necesidad), mejora del rendimiento en tareas especializadas donde el modelo base tiene conocimiento limitado, y reducción de coste cuando hay un volumen de inferencia muy alto (el modelo fine-tuneado con menor prompt puede ser más económico).

BLOQUE 2 · OBSERVABILIDAD Y MONITORIZACIÓN

Ver lo que está pasando: observabilidad de sistemas LLM

Las tres dimensiones de la observabilidad en LLMs

La observabilidad de los sistemas LLM tiene tres dimensiones: logs (registro de todas las interacciones: prompts, respuestas, metadatos de la llamada), trazas (seguimiento del flujo de una solicitud a través de todos los componentes del sistema: RAG retrieval, LLM call, herramientas, etc.), y métricas (agregaciones de los logs y trazas en el tiempo: latencia promedio, coste por solicitud, tasas de error, métricas de calidad del modelo). Sin estas tres dimensiones, operar un sistema LLM en producción es como conducir sin instrumentos: puedes ir hacia adelante pero no sabes si vas en la dirección correcta, a qué velocidad, o cuándo te vas a quedar sin combustible.

Las métricas de calidad del modelo son las más difíciles de monitorizar porque requieren evaluar la calidad del texto generado, no solo métricas técnicas (latencia, errores HTTP). Las métricas proxy (tasa de thumbs down de los usuarios, tasa de escalada a humanos, tasa de repetición de consultas que sugiere que la respuesta anterior no fue satisfactoria) pueden detectar degradación del modelo sin necesitar evaluación humana directa. LLM-as-judge (usar un segundo LLM para evaluar la calidad de las respuestas del sistema, sobre una muestra del 5-10% del tráfico) proporciona evaluaciones de calidad automáticas con mayor correlación con el juicio humano que las métricas proxy.

LangSmith, Langfuse, y el ecosistema de observabilidad

LangSmith (LangChain) es la herramienta de observabilidad más integrada con el ecosistema LangChain/LangGraph. Se activa con dos variables de entorno y traza automáticamente todas las llamadas a LLMs, todas las recuperaciones RAG, y todas las ejecuciones de herramientas, produciendo trazas visuales que muestran el flujo completo de cada solicitud. Langfuse es la alternativa open-source más popular, con soporte para múltiples frameworks y modelos, y la ventaja de poder desplegarse on-premise para mayor privacidad. Weights & Biases Traces integra la observabilidad de LLMs con el tracking de experimentos de entrenamiento, útil para equipos que hacen fine-tuning.

Alertas y respuesta a incidentes

Las alertas de producción para sistemas LLM deben cubrir: latencia (alertar si el P95 de latencia supera el SLA), coste (alertar si el gasto de tokens supera el presupuesto diario o si hay un pico anómalo de uso), calidad (alertar si la métrica de calidad del modelo cae por debajo del umbral, típicamente detectado con evaluación sobre una muestra continua), y seguridad (alertar si los filtros de contenido detectan un número anómalo de outputs problemáticos o patrones de uso sospechosos). El runbook de respuesta a incidentes para cada tipo de alerta debe estar definido antes de que ocurra el incidente, no durante el mismo.

BLOQUE 3 · CI/CD Y TEST-TIME COMPUTE

Automatizar la calidad: CI/CD para sistemas de IA

El pipeline de CI/CD para LLMs

El CI/CD para sistemas basados en LLMs extiende el CI/CD estándar de software con checks específicos para IA. El pipeline de PR en un sistema LLM incluye: tests unitarios del código Python del sistema (pytest), tests de integración de las herramientas del agente (verificar que las funciones de herramienta manejan correctamente los casos de borde), tests de seguridad del prompt (el golden dataset adversarial: el sistema debe rechazar definitivamente estos prompts maliciosos), tests de calidad del LLM (el golden dataset de casos de uso representativos: el sistema debe mantener o mejorar el rendimiento anterior), y análisis de costes (verificar que el cambio no aumenta el coste de tokens por encima de un umbral). Solo si todos estos checks pasan, el PR puede mergearse.

El test suite de evaluación del LLM es el componente más crítico y el más diferente al CI/CD estándar. Requiere: un golden dataset de 50-200 casos representativos con outputs de referencia (actualizados periódicamente con nuevos tipos de inputs observados en producción), evaluadores automáticos para medir la calidad (LLM-as-judge con rúbricas específicas para el caso de uso), y umbrales de regresión (si el rendimiento cae más de X% respecto a la versión anterior, el pipeline falla). Este enfoque previene las regresiones accidentales de calidad que son una de las causas más comunes de degradación de sistemas LLM en producción.

Test-Time Compute y modelos de razonamiento

Test-time compute es el paradigma de mejorar la calidad del modelo invirtiendo más computación durante la inferencia (en el tiempo de uso, no en el tiempo de entrenamiento). Los modelos de razonamiento como o3 de OpenAI y Claude con extended thinking usan este paradigma: en lugar de generar directamente la respuesta, generan un "scratchpad" de razonamiento (que puede ser muy extenso) antes de producir la respuesta final. Este razonamiento adicional en el tiempo de inferencia produce mejoras significativas en benchmarks de matemáticas, código, y razonamiento lógico. La implicación para LLMOps es que hay ahora un trade-off explícito entre coste/latencia y calidad que se puede ajustar en tiempo de ejecución: para preguntas sencillas, usar el modelo estándar (rápido y barato); para preguntas complejas, usar el modelo de razonamiento (más lento y caro pero más preciso).

RESUMEN EJECUTIVO DEL MÓDULO 8: LLMOps cierra la brecha entre experimentos y sistemas en producción. El 87% de los proyectos de IA nunca llegan a producción (VentureBeat). Las prácticas clave son: versionado de modelos + prompts + datos (DVC + git + Model Registry), decisión de fine-tuning vs prompting (siempre empieza con prompting), observabilidad completa (LangSmith o Langfuse desde el día 1: logs + trazas + métricas), CI/CD con golden dataset adversarial y de calidad, y gestión de costes de tokens (prompt caching, modelos escalonados por complejidad). Test-time compute con modelos de razonamiento: trade-off

explícito coste/latencia vs calidad ajustable en runtime. El monitoring en producción es tan importante como el testing previo al despliegue: los sistemas LLM se degradan por cambios en la distribución de los inputs, por actualizaciones del modelo del proveedor, y por cambios en los datos del corpus RAG.